

BÀI THỰC HÀNH COMMAND PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React
 - ✓ Áp dụng Command Pattern (GoF)
 - ✓ Có Undo (rollback thao tác Order)
-

BÀI TOÁN

Hệ thống **Order Management**:

Người dùng có thể:

- Tạo đơn hàng
- Thêm sản phẩm vào đơn
- Xoá sản phẩm khỏi đơn
- Xác nhận đơn hàng

👉 Yêu cầu:

- Không gọi trực tiếp logic Order
 - Mọi thao tác phải thông qua **Command Pattern**
 - Có thể **Undo thao tác gần nhất**
-

KIẾN TRÚC

```
React (Client)
  ↓
POST /api/orders/execute
  ↓
OrderController
  ↓
CommandInvoker
  ↓
ICommand
  ↓   ↓   ↓
CreateOrderCommand
AddItemCommand
RemoveItemCommand
ConfirmOrderCommand
  ↓
Receiver (OrderService)
```

BACKEND – ASP.NET CORE WEB API

Tạo project

```
dotnet new webapi -n CommandPatternDemo
cd CommandPatternDemo
```

Cấu trúc

```
CommandPatternDemo
├── Controllers
│   └── OrderController.cs
├── Commands
│   ├── ICommand.cs
│   ├── CreateOrderCommand.cs
│   ├── AddItemCommand.cs
│   ├── RemoveItemCommand.cs
│   └── ConfirmOrderCommand.cs
├── Receivers
│   └── OrderService.cs
├── Invokers
│   └── CommandInvoker.cs
├── Models
│   ├── Order.cs
│   ├── OrderItem.cs
│   └── CommandRequest.cs
└── Program.cs
```

1. COMMAND INTERFACE

```
public interface ICommand
{
    void Execute();
    void Undo();
}
```

2. RECEIVER – ORDER SERVICE

```
public class OrderService
{
    public Order CurrentOrder { get; private set; } = new
    Order();
}
```

```

public void CreateOrder()
{
    CurrentOrder = new Order();
    Console.WriteLine("Order created");
}

public void AddItem(string product, int qty)
{
    CurrentOrder.Items.Add(new OrderItem { Product =
product, Quantity = qty });
    Console.WriteLine($"Added {product}");
}

public void RemoveItem(string product)
{
    CurrentOrder.Items.RemoveAll(x => x.Product ==
product);
    Console.WriteLine($"Removed {product}");
}

public void ConfirmOrder()
{
    CurrentOrder.IsConfirmed = true;
    Console.WriteLine("Order confirmed");
}
}

```

✿ 3. CONCRETE COMMANDS

Commands/ CreateOrderCommand.cs

```

public class CreateOrderCommand : ICommand
{
    private readonly OrderService _service;

    public CreateOrderCommand(OrderService service)
    {
        _service = service;
    }

    public void Execute()
    {
        _service.CreateOrder();
    }
}

```

```
    public void Undo()
    {
        _service.CurrentOrder = null;
    }
}
```

Commands/AddItemCommand.cs

```
public class AddItemCommand : ICommand
{
    private readonly OrderService _service;
    private readonly string _product;
    private readonly int _qty;

    public AddItemCommand(OrderService service, string
product, int qty)
    {
        _service = service;
        _product = product;
        _qty = qty;
    }

    public void Execute()
    {
        _service.AddItem(_product, _qty);
    }

    public void Undo()
    {
        _service.RemoveItem(_product);
    }
}
```

Commands/RemoveItemCommand

```
public class RemoveItemCommand : ICommand
{
    private readonly OrderService _service;
    private readonly string _product;
    private OrderItem _backup;

    public RemoveItemCommand(OrderService service, string
product)
    {
        _service = service;
```

```

        _product = product;
    }

    public void Execute()
    {
        _backup = _service.CurrentOrder.Items
            .FirstOrDefault(x => x.Product == _product);

        _service.RemoveItem(_product);
    }

    public void Undo()
    {
        if (_backup != null)
            _service.CurrentOrder.Items.Add(_backup);
    }
}

```

Commands/ ConfirmOrderCommand.cs

```

public class ConfirmOrderCommand : ICommand
{
    private readonly OrderService _service;

    public ConfirmOrderCommand(OrderService service)
    {
        _service = service;
    }

    public void Execute()
    {
        _service.ConfirmOrder();
    }

    public void Undo()
    {
        _service.CurrentOrder.IsConfirmed = false;
    }
}

```

4. INVOKER

```

public class CommandInvoker
{
    private Stack<ICommand> _history = new();
}

```

```

public void ExecuteCommand(ICommand command)
{
    command.Execute();
    _history.Push(command);
}

public void Undo()
{
    if (_history.Count > 0)
    {
        var cmd = _history.Pop();
        cmd.Undo();
    }
}
}

```

5. MODEL

Models/Order.cs

```

public class Order
{
    public List<OrderItem> Items { get; set; } = new();
    public bool IsConfirmed { get; set; }
}

```

Models/OrderItem.cs

```

public class OrderItem
{
    public string Product { get; set; }
    public int Quantity { get; set; }
}

```

Models/CommandRequest.cs

```

public class CommandRequest
{
    public string Action { get; set; }
    public string Product { get; set; }
    public int Quantity { get; set; }
}

```

6. CONTROLLER

```

using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/orders")]
public class OrderController : ControllerBase
{
    private readonly CommandInvoker _invoker;
    private readonly OrderService _service;

    public OrderController(CommandInvoker invoker,
OrderService service)
    {
        _invoker = invoker;
        _service = service;
    }

    [HttpPost("execute")]
    public IActionResult Execute(CommandRequest request)
    {
        ICommand? command = request.Action switch
        {
            "create" => new CreateOrderCommand(_service),
            "add" => new AddItemCommand(_service,
request.Product, request.Quantity),
            "remove" => new RemoveItemCommand(_service,
request.Product),
            "confirm" => new
ConfirmOrderCommand(_service),
            _ => null
        };

        if (command == null)
        {
            return BadRequest("Invalid action");
        }

        _invoker.ExecuteCommand(command);
        return Ok();
    }

    [HttpPost("undo")]
    public IActionResult Undo()
    {
        _invoker.Undo();
        return Ok();
    }
}

```

7. Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();

// Dependency Injection
builder.Services.AddSingleton<CommandInvoker>();
builder.Services.AddSingleton<OrderService>();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// CORS for React frontend
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();

// Configure middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseCors("AllowReact");

app.MapControllers();

app.Run();
```


❶ Tạo project

```
npx create-react-app command-ui  
cd command-ui  
npm install axios
```

❷ GIAO DIỆN CHECKOUT

```
import axios from "axios";  
  
function App() {  
  const send = async (action, product = null, quantity =  
null) => {  
    try {  
      const payload = { action };  
  
      if (product !== null) payload.product = product;  
      if (quantity !== null) payload.quantity = quantity;  
  
      const response = await  
axios.post("/api/orders/execute", payload);  
      console.log("Success:", response.data);  
      alert("Command executed successfully");  
    } catch (error) {  
      console.error("Execute command failed:", error);  
      alert("Command execution failed");  
    }  
  };  
  
  const undo = async () => {  
    try {  
      const response = await  
axios.post("/api/orders/undo");  
      console.log("Undo success:", response.data);  
      alert("Undo successful");  
    } catch (error) {  
      console.error("Undo failed:", error);  
      alert("Undo failed");  
    }  
  }  
}
```

```

    };

    return (
      <div style={{ padding: 40 }}>
        <h2>Order Management</h2>

        <button onClick={() => send("create")}>Create
Order</button>
        <button onClick={() => send("add", "Laptop",
1)}>Add Laptop</button>
        <button onClick={() => send("remove",
"Laptop")}>Remove Laptop</button>
        <button onClick={() =>
send("confirm")}>Confirm</button>
        <button onClick={undo}>UNDO</button>
      </div>
    );
  }

export default App;

```

LƯỖNG XỬ LÝ

Add Item

```

React → Controller
      ↓
AddItemCommand
      ↓
Invoker.Execute
      ↓
OrderService.AddItem

```

Undo

```

Invoker.Pop()
      ↓
Command.Undo()
      ↓
Rollback Order

```

KIẾN THỨC ĐẠT ĐƯỢC

- ✓ Áp dụng Command Pattern trong domain thực tế (Order)
 - ✓ Hiểu Undo = reverse business logic
 - ✓ Tách biệt rõ:
 - Controller (Client)
 - Invoker
 - Command
 - Receiver (Domain Service)
 - ✓ Chuẩn bị cho:
 - Transaction processing
 - Event sourcing
 - CQRS / Command Bus
-

MỞ RỘNG

1. Thêm **Redo stack**
2. Lưu command history vào DB
3. Áp dụng **CQRS (Command vs Query)**
4. Dùng **MediatR / Command Bus**
5. Event sourcing cho Order